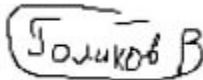


Министерство науки и высшего образования Российской Федерации Федеральное
государственное автономное образовательное учреждение высшего образования
“Санкт-Петербургский политехнический университет Петра Великого”
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

Отчет по лабораторной работе № 1.4
по дисциплине
«Системы управления базами данных»

Выполнил студент гр.
5130904/30107

подпись: 

Голиков В.С.

Руководитель

подпись: _____

Прокофьев О.В.

Санкт-Петербург
2026

Оглавление

Постановка задачи.....	3
Выполнение работы.....	4
1. READ COMMITTED.....	5
1.1 Lost Update.....	5
1.2 Dirty Read.....	6
1.3 Non-Repeatable Read.....	6
1.4 Phantoms.....	7
2. REPEATABLE READ.....	8
2.1 Non-Repeatable Read.....	8
2.2 Phantom.....	9
2.3 Serialization.....	10
3. SERIALIZABLE.....	12

Постановка задачи

Практическое задание посвящено контролю целостности данных, который производится с помощью механизма транзакций и триггеров. Транзакции позволяют рассматривать группу операций как единое целое, либо отрабатывают все операции, либо ни одной. Это позволяет избегать несогласованности данных. Триггеры позволяют проверять целостность данных в момент выполнения транзакций, поддерживать целостность, внося изменения, и откатывать транзакции, приводящие к потере целостности. Необходимо подготовить SQL-скрипты для проверки наличия аномалий (потерянных изменений, грязных чтений, неповторяющихся чтений, фантомов) при параллельном исполнении транзакций на различных уровнях изолированности SQL/92 (READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE). Подготовленные скрипты должны работать с таблицей, содержащей 3-4 атрибута и до 30 строк. Для проверки наличия аномалий потребуются два параллельных сеанса, операторы в которых выполняются пошагово:

- Установить в обоих сеансах уровень изоляции READ COMMITTED/READ UNCOMMITTED (для postgresql это синонимы). Выполнить сценарии проверки наличия аномалий потерянных изменений, грязных чтений, неповторяющихся чтений, фантомов.
- Установить в обоих сеансах уровень изоляции REPEATABLE READ. Выполнить сценарии проверки наличия аномалий неповторяющихся чтений и фантомов (проверить те же скрипты, что и для READ COMMITTED), сериализации.
- Установить в обоих сеансах уровень изоляции SERIALIZABLE. Выполнить сценарий проверки фиксации аномалии сериализации (проверить тот же скрипт аномалии сериализации, что и для REPEATABLE READ).

Выполнение работы

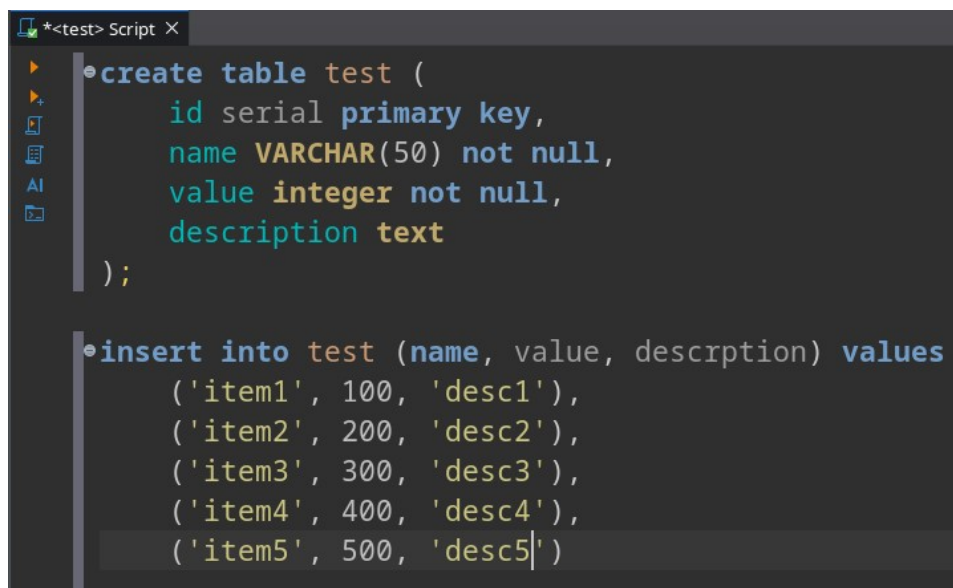
Стандарт SQL/92 определяет 4 уровня изоляции, которые защищают (или не защищают) от определенных аномалий:

Уровень изоляции	Lost Update	Dirty Read	Non-Repeatable Read	Phantom
READ UNCOMMITTED	-	-	-	-
READ COMMITTED	+	+	-	-
REPEATABLE READ	+	+	+	-
SERIALIZABLE	+	+	+	+

Для PostgreSQL:

- **READ UNCOMMITTED** - работает аналогично **READ COMMITTED** (Dirty Read невозможно)
- **REPEATABLE READ** - защищает от фантомов
- **SERIALIZABLE** — использует более строгий механизм слежения за конфликтами

Создадим тестовую таблицу test с случайными данными:



```
*<test> Script X
create table test (
  id serial primary key,
  name VARCHAR(50) not null,
  value integer not null,
  description text
);

insert into test (name, value, description) values
('item1', 100, 'desc1'),
('item2', 200, 'desc2'),
('item3', 300, 'desc3'),
('item4', 400, 'desc4'),
('item5', 500, 'desc5');
```

Результатом данного скрипта будет прстейшая таблица с 5-ю строчками

	123 id	AZ name	123 value	AZ description
1	1	item1	100	desc1
2	2	item2	200	desc2
3	3	item3	300	desc3
4	4	item4	400	desc4
5	5	item5	500	desc

Теперь начнем эксперименты по заданию.

1. READ COMMITTED

1.1 Lost Update

Шаг	Терминал А	Терминал Б
1	begin; set transaction isolation level read committed; select value from test where id = 1; результат: 100	
2		begin; set transaction isolation level read committed; select value from test where id = 1; результат: 100
3	update test set value = value + 50 where id = 1; ожидаем 150 после commit	
4		update test set value = value + 30 where id = 1; ожидаем 180 после commit
5	Commit	Commit

В результате выполнения всех команд в разных терминалах получили $100 + 50 + 30 = 180$. Следовательно, потеря изменений не произошла.

1.2 Dirty Read

Шаг	Терминал А	Терминал Б
1	begin; set transaction isolation level read committed; update test set value = 999 where id = 2 результат: перезапишем 999	
2		begin; set transaction isolation level read committed; select value from test where id = 2 результат: 200
3	Rollback;	Commit;

На READ COMMITTED грязное чтение невозможно, это видно по результатам работы Терминала Б

1.3 Non-Repeatable Read

Шаг	Терминал А	Терминал Б
1	begin; set transaction isolation level read committed; select value from test where id = 3; результат: 300	
2		begin; set transaction isolation level read committed; update test set value = 333 where id = 3; commit; результат: перезапишем 333
3	Select value from test where id = 3; результат: 333	

В результате выполнения всех команд, второй терминал изменил значение элемента с id 3 на 333 и первый терминал увидел это изменение. Здесь проявляется аномалия неповторяющегося чтения

1.4 Phantoms

Шаг	Терминал А	Терминал Б
1	<pre>begin; set transaction isolation level read committed; select count(*) from test where value > 400; результат: 1</pre>	
2		<pre>begin; set transaction isolation level read committed; insert into test (name, value, description) values ('phantom', 600, 'pahntom desc'); commit; результат: создали новую запись</pre>
3	<pre>select count(*) from test where value > 400; результат: 2</pre>	

Наблюдается увеличение счетчика, то есть мы посчитали нашу фантомную запись. Аномалия проявляется.

2. REPEATABLE READ

2.1 Non-Repeatable Read

Шаг	Терминал А	Терминал Б
1	begin; set transaction isolation level repeatable read; select value from test where id = 3; результат: 300	
2		begin; set transaction isolation level repeatable read; update test set value = 777 where id = 3; commit; результат: перезапишем 777
3	Select value from test where id = 3; результат: 300	
4	Commit; select value from test where id = 3; результат: 777	

В середине транзакции на терминале А изменений из терминала Б не видно, а после комита — они видны. Аномалия не проявляется

2.2 Phantom

Шаг	Терминал А	Терминал Б
1	<pre>begin; set transaction isolation level repeatable read; select count(*) from test where value > 400; результат: 2</pre>	
2		<pre>begin; set transaction isolation level repeatable read; insert into test (name, value, description) values ('phantom', 600, 'pahntom desc'); commit; результат: создали новую запись</pre>
3	<pre>select count(*) from test where value > 400; результат: 2</pre>	
4	<pre>Commit; select count(*) from test where value > 400; результат: 3</pre>	

REPEATABLE READ защитил данные от фантомов: внутри транзакции на терминале А изменений из терминала Б нет, а после комита число изменилось. Аномалия не проявляется

2.3 Serialization

Шаг	Терминал А	Терминал Б
1	begin; set transaction isolation level repeatable read;	begin; set transaction isolation level repeatable read;
2	Select sum(value) from test where id in (1,2) результат: 250	Select sum(value) from test where id in (3,8) результат: 400
3	insert into test (name, value, description) values ('item6', 250, 'desc6');	insert into test (name, value, description) values ('item7', 400, 'desc7');
4	Commit; результат: успешная фиксация	Commit; результат: успешная фиксация

Параллельно выполняя инструкции мы не достигли такого же результата, как если бы выполняли их последовательно. Поскольку мы находимся на уровне изоляции REPEATABLE READ, обе транзакции успешно зафиксировались, но аномалия сериализации все равно проявляется.

Если же использовать команду изменения данных и зафиксировать изменения в первой транзакции, то вторая, параллельная транзакция, которая ссылается на те же значения, получит откат с ошибкой, приведенной ниже на двух скриншотах (Терминал А и Терминал Б соответственно)

```
test=# begin;  
BEGIN  
test=# set transaction isolation level repeatable read;  
SET  
test=# Update test set value = value - 50 where id = 1;  
UPDATE 1  
test=# commit;  
COMMIT
```

```
test=# begin;  
BEGIN  
test=# set transaction isolation level repeatable read;  
SET  
test=# Update test set value = value + 30 where id = 1;  
ERROR:  could not serialize access due to concurrent update  
test=!#  
test=!# rollback;  
ROLLBACK
```

3. SERIALIZABLE

Шаг	Терминал А	Терминал Б
1	begin; set transaction isolation level repeatable read;	begin; set transaction isolation level repeatable read;
2	Select sum(value) from test where id in (1,2) результат: 250	Select sum(value) from test where id in (3,8) результат: 400
3	insert into test (name, value, description) values ('item6', 250, 'desc6');	insert into test (name, value, description) values ('item7', 400, 'desc7');
4	Commit; результат: успешная фиксация	Commit; результат: ошибка

На уровне изоляции SERIALIZABLE скрипт с предыдущего уровня выполнялся с ошибкой на терминале Б. Как говорилось ранее, на этом уровне изоляции моделируется последовательное выполнение всех зафиксированных транзакций, как если бы они выполнялись одна за другой. В данном примере наглядно показано, что при параллельной работе двух транзакций все выполняется, однако при попытке зафиксироваться, фиксируется только одна из транзакций, а вторая получает откат с выводом ошибки (представлена на скриншоте ниже из терминала Б)

```
test=# begin;
BEGIN
test=# set transaction isolation level serializable;
SET
test=# select sum(value) from test where id in (3,4);
 sum
-----
 1177
(1 row)

test=# select sum(value) from test where id in (4,8);
 sum
-----
 400
(1 row)

test=# insert into test (name, value, description) values (
test( 'item7', 400, 'desc7');
INSERT 0 1
test=# commit;
ERROR:  could not serialize access due to read/write dependencies among transactions
DETAIL:  Reason code: Canceled on identification as a pivot, during commit attempt.
```